DETAILED TECHNICAL REPORT AND MANUAL

BIPE Academic Portal

A combined technical report, implementation handbook, process manual, and deployment-oriented reference for the role-based academic management portal built with PHP, MySQL, modular assets, and server-rendered workflows.

PREPARED FOR

Project documentation, academic submission, maintenance, and onboarding use

APPLICATION ROLES

Administrator, Faculty, Student

TECHNOLOGY

PHP, MySQL, PDO, JavaScript, modular CSS, session-based authentication

DOCUMENT DATE

5 April 2026

Public-safe edition. This document intentionally excludes passwords, live credentials, private contact data, and deployment secrets.

DOCUMENT CONTROL

Document Identity

PROJECT NAME	BIPE Academic Portal	DOCUMENT TYPE	Detailed Technical Report and User Manual
APPLICATION STYLE	Role-based multi-page web portal	PRIMARY AUDIENCE	Project evaluators, maintainers, administrators, onboarding developers
ROLES COVERED	Admin, Faculty, Student	DATABASE	MySQL with normalized operational tables
RUNTIME STACK	PHP 8+, PDO, vanilla JS, modular CSS	CONFIDENTIALITY	Credential-free public-safe documentation

Table of Contents

1. Executive Summary
2. Non-Technical Overview
3. Project Scope and Stakeholders
4. Functional and Non-Functional Requirements
5. System Architecture and Technology Stack
6. Process Flow Diagrams
7. Data Flow Diagrams
8. Database Design and Data Dictionary
9. Normalization Review
10. Security and Governance Controls
11. Operational Flows
12. Role-wise User Manual
13. Backup, Restore, and Maintenance
14. Testing, Risks, and Troubleshooting
15. Recommended Enhancements
16. Conclusion

EXECUTIVE SUMMARY

Project Purpose

BIPE Academic Portal is a centralized web application designed to digitize institution-level academic operations. It brings together admission-linked student activation, faculty onboarding and approval, attendance capture, marks publishing, assignment tracking, support workflows, reporting, and administrative governance inside one portal.

3
core user roles with separate dashboards and permissions

18+
persistent tables supporting academic and operational workflows

1
shared bootstrap and service layer coordinating all modules

100%
credential-free documentation intent for safe sharing

Why This Portal Matters

Operational Consolidation

Attendance, marks, assignments, approvals, and report cards are managed inside one system instead of fragmented spreadsheets and informal messages.

Role Clarity

Administrators govern the institution, faculty update academic records, and students consume their own information through a restricted self-service interface.

Institutional Control

Audit logging, mark locks, support request review, and backup and restore tooling improve accountability, recovery readiness, and process discipline.

Important implementation note

The portal already includes OTP-based password reset data structures and workflow methods. However, external reset delivery methods are currently stubs and must be connected to a mail or notification provider before production rollout.

NON-TECHNICAL VIEW

Business Problem and Solution Narrative

Educational institutions frequently struggle with disconnected academic records. Student data may live in spreadsheets, attendance may be recorded by each faculty member in their own format, marks may be shared through separate files, and important requests such as faculty approval or account recovery may remain dependent on personal communication. This creates duplication, delay, inconsistency, and weak auditability.

The BIPE Academic Portal addresses this by converting the institution's routine processes into controlled digital workflows. Students activate only pre-seeded roster records, faculty accounts move through approval status, class-wise attendance is stored session by session, marks are published against controlled mark types, and support requests are reviewed through an administrative queue. The result is a more transparent, faster, and more measurable academic operations environment.

Stakeholder Matrix

STAKEHOLDER	PRIMARY OBJECTIVE	TYPICAL PORTAL ACTIONS	VALUE RECEIVED
Administrator	Govern academic data, accounts, and institutional controls	Manage students, approve faculty, review requests, backup data, lock marks, run reports	Centralized oversight and operational traceability
Faculty	Maintain timely class records	Record attendance, upload marks, track assignments, review reports, update profile	Faster routine work with less duplication
Student	Access personal academic information	Activate account, view attendance, view marks, review assignments, update contact profile	Better visibility into progress and expectations
Institution Management	Track academic performance and compliance	Use reports, audit trails, backups, and report cards	Decision support and control over record quality

Project Scope

- Student roster management
- Faculty approval workflow
- Subject and semester setup
- Attendance sessions and history
- Marks entry and CSV import
- Assignment tracking
- Report cards and analytics
- Support requests, feedback, issues
- Audit, backup, restore, reset operations

REQUIREMENTS

Functional and Non-Functional Requirements

Functional Requirements

- Authenticate administrators, faculty members, and students with role-specific access.
- Allow faculty registration with approval gates before dashboard access.
- Allow students to activate only existing seeded roster records.
- Support student CRUD operations and CSV-based bulk import.
- Support subject import per department, year, and semester.
- Create, edit, and review attendance sessions with class filters and remarks.
- Capture marks manually or through CSV templates with max-mark validation.
- Track assignment status by subject, semester, and label.
- Generate class reports, report cards, and department-level analytics.
- Handle support requests across request, feedback, and issue categories.
- Record audit logs for sensitive and administrative actions.
- Export and restore full data backups.

Non-Functional Requirements

- Secure session handling, CSRF protection, and defensive HTTP headers.
- Usable on both desktop and mobile through responsive layouts.
- Maintainable through modular page-specific CSS and shared helper layers.
- Recoverable through exportable backups and bounded reset operations.
- Data consistency enforced through foreign keys, unique constraints, and validation rules.
- Auditability through operational logging and approval state transitions.
- Performance suitable for institution-scale departmental workflows using indexed relational tables.
- Deployable with environment-based configuration and minimal infrastructure complexity.

What makes the design practical

The system balances simplicity and governance. It remains a straightforward server-rendered PHP application, but still includes approvals, locks, audit trails, rate limits, backup support, and role boundaries that are often missing in small academic portals.

ARCHITECTURE

System Architecture and Technology Stack

The project follows a layered server-rendered architecture. Role-specific pages live under `admin/`, `faculty/`, and `student/`. Shared runtime concerns such as configuration, database access, authentication, helper utilities, layout rendering, error handling, and business logic are centralized in the `app/` directory.

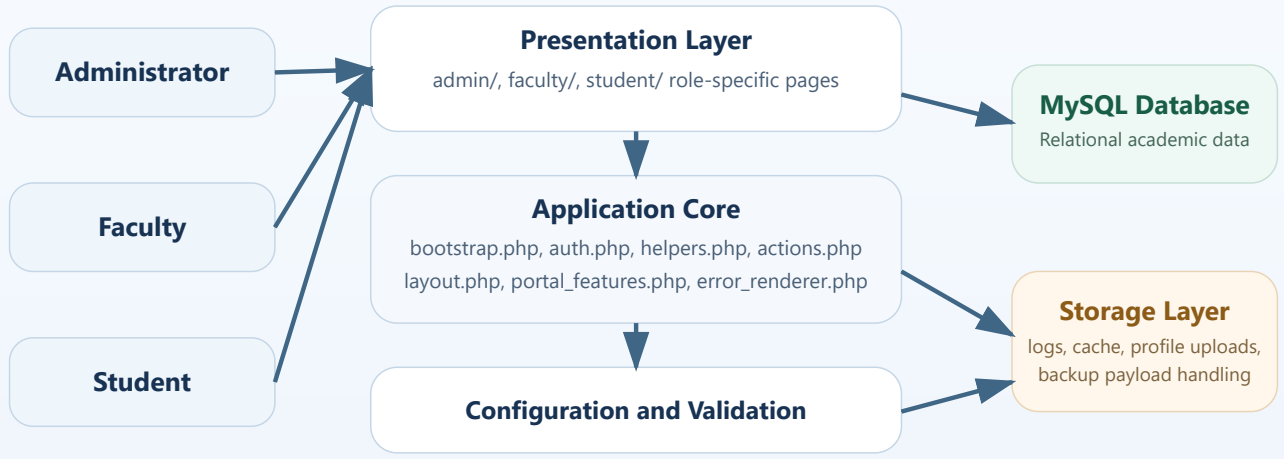

Figure 1. Role-based presentation pages invoke a shared application core, which persists data in MySQL and uses file-based runtime storage for logs, cache, and uploads.

Frontend

Server-rendered HTML with modular CSS and lightweight JavaScript for page-specific interaction.

Backend

PHP procedures and helper functions using PDO with exception-based database handling.

Persistence

Normalized MySQL schema with foreign keys, unique constraints, and audit-oriented supporting tables.

FLOW DIAGRAMS

Core Process Flow Diagrams

Access and Approval Flow

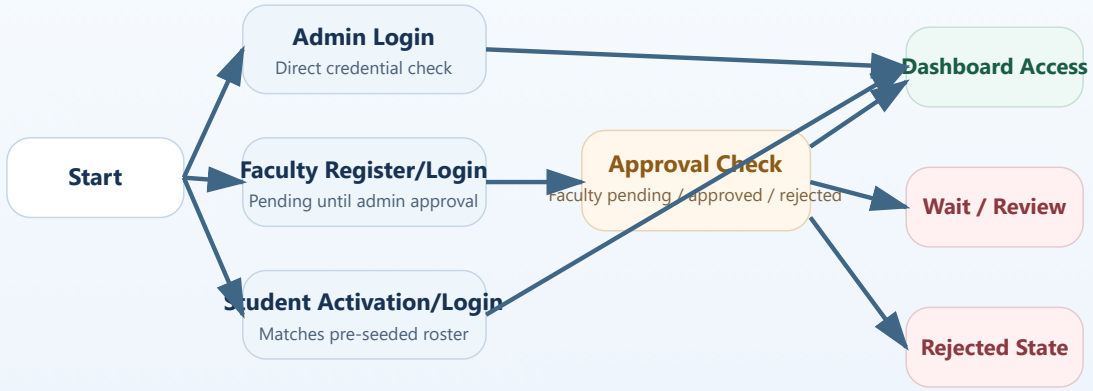

Figure 2. Access is role-dependent. Faculty flows are stateful because approval status directly gates dashboard entry.

Attendance, Marks, and Assignments Flow

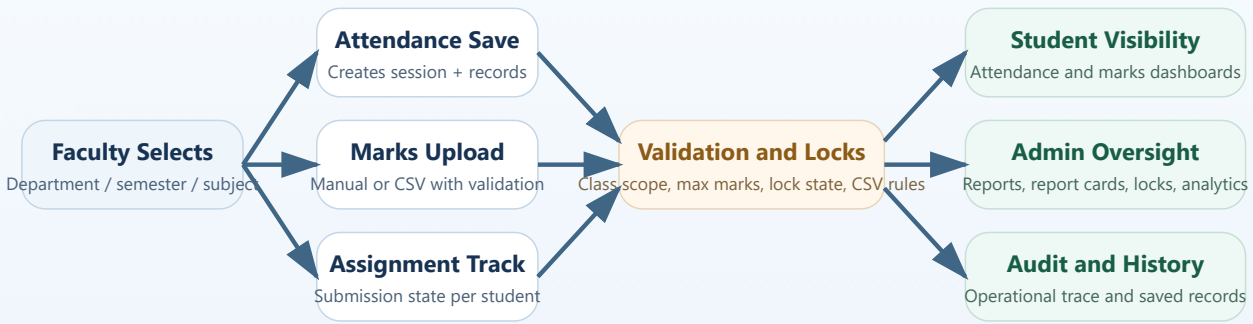

Figure 3. Faculty operational inputs pass through validation and lock controls before becoming visible to students and administrators.

DATA FLOW

Data Flow Diagrams

DFD Level 0: Context Diagram

Figure 4. The portal acts as the primary processing system between external actors and the central academic database.

DFD Level 1: Major Internal Processes

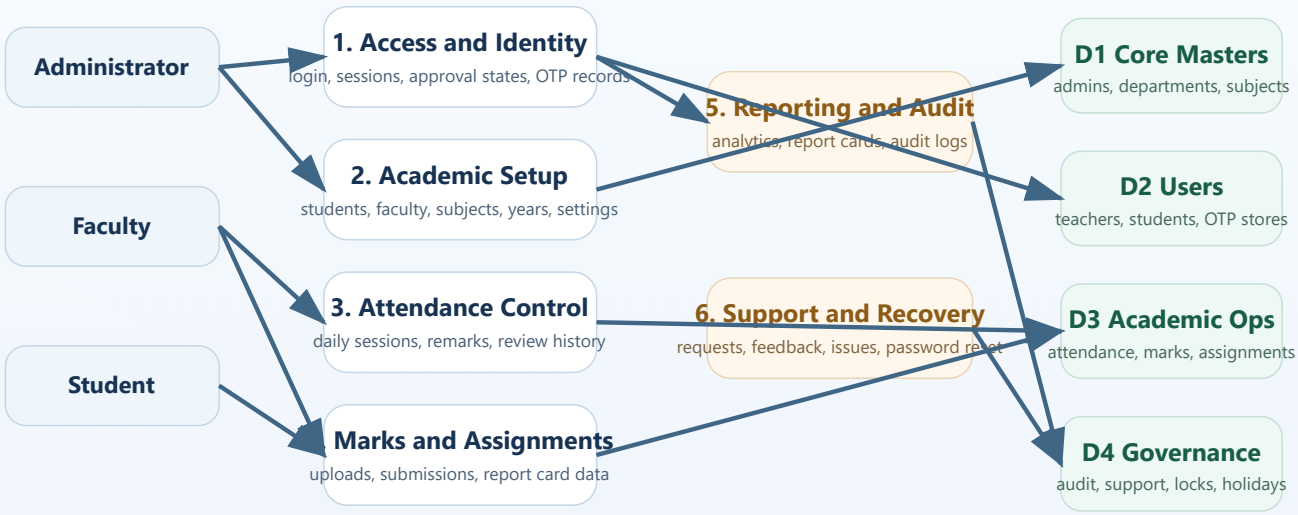

Figure 5. Major internal processes are separated into access, setup, academic operations, reporting, and support functions against grouped data stores.

DATA MODEL

Database Design and Data Dictionary

The database is designed around master entities, user entities, academic transaction entities, and governance-support entities. Surrogate integer primary keys are used throughout, while important business uniqueness rules are preserved via unique indexes.

Entity Grouping

Master and Reference Data

- `academic_years`
- `departments`
- `subjects`
- `mark_types`
- `mark_locks`

User and Control Data

- `admins`
- `teachers`
- `students`
- `admin_otp_requests`
- `password_reset_otps`
- `support_requests`

Academic Transaction Data

- `attendance_sessions` and `attendance_records`
- `mark_uploads` and `mark_records`
- `assignments` and `assignment_submissions`

Governance and Operational Data

- `holiday_events`
- `audit_logs`

Key Data Dictionary

TABLE	PURPOSE	PRIMARY KEY	REPRESENTATIVE COLUMNS	IMPORTANT RELATIONS
`academic_years`	Current and historical academic sessions	id	label, start_date, end_date, is_current	Referenced by attendance, marks, assignments, holidays

TABLE	PURPOSE	PRIMARY KEY	REPRESENTATIVE COLUMNS	IMPORTANT RELATIONS
`admins`	Administrator accounts	id	username, full_name, email, password_hash	Linked to support request reviews and OTP resets
`departments`	Department master data	id	name, code, short_name, theme_color	Referenced by teachers, students, subjects, locks
`teachers`	Faculty accounts with approval states	id	teacher_code, email, status, registered_at	Belongs to department; referenced by attendance, marks, assignments
`students`	Student roster and activation state	id	enrollment_no, department_id, semester_no, password_hash	Belongs to department; referenced by attendance, marks, submissions
`subjects`	Semester-specific course definitions	id	department_id, semester_no, subject_code, subject_name	Referenced by marks, assignments, report cards
`mark_types`	Assessment type catalog	id	label, max_marks, sort_order	Used by marks workflow to snapshot exam types
`mark_locks`	Administrative control over mark editing	id	department_id, semester_no, locked_at	Guards marks save operations
`support_requests`	Unified request, feedback, and issue queue	id	category, requester_role, request_type, status	Reviewed by admins; supports approval pipeline
`holiday_events`	Holiday, leave, and festival calendar records	id	scope_type, event_date, end_date, event_type, title	Used by attendance workflows

TABLE	PURPOSE	PRIMARY KEY	REPRESENTATIVE COLUMNS	IMPORTANT RELATIONS
`attendance_sessions`	One attendance header per class-date scope	id	department_id, year_level, semester_no, attendance_date	Parent for attendance_records
`attendance_records`	Student attendance line items	id	attendance_session_id, student_id, status	Many-to-one to sessions and students
`mark_uploads`	Header record for one subject/exam type upload	id	department_id, semester_no, subject_id, exam_type, max_marks	Parent for mark_records
`mark_records`	Student marks under one upload	id	mark_upload_id, student_id, marks_obtained, is_absent	Many-to-one to uploads and students
`assignments`	Assignment headers per subject and label	id	subject_id, assignment_label, due_date, notes	Parent for assignment_submissions
`assignment_submissions`	Submission state per student	id	assignment_id, student_id, submission_status	Links assignments to students
`audit_logs`	Operational activity trace	id	actor_role, actor_identifier, action_name, metadata_json	Supports governance and diagnosis

Entity Relationship Overview

```
departments
  |--< teachers
  |--< students
  |--< subjects
  |--< mark_locks

academic_years
  |--< attendance_sessions --< attendance_records >-- students
  |--< mark_uploads       --< mark_records       >-- students
```

```
  |--< assignments        --< assignment_submissions >-- students
  |--< holiday_events

subjects
  |--< mark_uploads
  |--< assignments

admins
  |--< admin_otp_requests
  |--< support_requests (reviewed_by_admin_id)

teachers
  |--< attendance_sessions
  |--< mark_uploads
  |--< assignments
  |--< holiday_events (creator)
```

NORMALIZATION

Normal Forms and Relational Quality Review

The schema largely follows third normal form principles, with a few deliberate snapshot-oriented denormalizations used for historical stability and reporting convenience.

NORMAL FORM	HOW THE PORTAL SATISFIES IT	EXAMPLES	OBSERVATIONS
1NF	Atomic fields, scalar values, no repeating groups inside single columns	`students`, `teachers`, `subjects`, `support_requests`	Attendance and marks line items are not stored as comma-separated lists; they are placed in child tables.
2NF	Attributes depend on the whole key, especially in many-to-many bridge-like tables	`attendance_records`, `mark_records`, `assignment_submissions`	Composite business uniqueness is enforced through unique indexes while surrogate keys simplify joins.
3NF	Descriptive attributes are stored in their own master tables instead of repeating across transactions	`departments`, `subjects`, `mark_types`	Transitive dependencies are mostly removed. Operational tables reference master rows by foreign key.
BCNF tendency	Most determinant columns are candidate keys or protected by uniqueness rules	`subjects`, `mark_locks`	Small pragmatic deviations exist where snapshots are intentionally preserved for audit or historical consistency.

Intentional Practical Denormalizations

Marks Snapshot Design

The marks workflow uses `mark_types` as a source of truth when saving data, but stores `exam_type` and `max_marks` directly in `mark_uploads`. This preserves the historical meaning of a marks upload even if the mark type catalog later changes.

Support Request Snapshot Fields

`support_requests` stores requester name, identifier, and email text snapshots. This avoids losing context if the originating user record changes later and supports review of requests submitted by guests or partially registered users.

Normalization conclusion

The schema is well suited to transactional academic data. It is normalized enough to avoid duplication-driven inconsistency, yet practical enough to preserve historical snapshots where auditability matters.

SECURITY

Security and Governance Controls

Implemented Controls

- Session hardening with strict mode, HttpOnly cookies, and inactivity timeout.
- CSRF token generation and automatic POST verification in the bootstrap layer.
- Security headers including frame denial, content type protections, and content security policy.
- Password hashing with `password_hash` and password strength validation.
- Upload validation for images, CSV, and JSON backup files.
- Rate-limit support backed by file-based storage.
- Exception logging and safe error rendering.
- Audit logging for sensitive operations such as approvals, marks save, and profile changes.

Governance Features

- Faculty approval states: pending, approved, rejected.
- Administrative review of support requests, feedback, and issues.
- Mark locking by department and semester.
- Backup export and restore with full-dataset overwrite capability.
- Academic-year rollover and reset modes for institutional maintenance.
- Filtered bulk deletion for student administration where explicitly invoked.

Security Considerations for Production

- Move all secrets to environment variables and remove fallback defaults that should not persist in production.
- Connect OTP delivery methods to an actual mail or notification service.
- Limit destructive admin actions with confirmation, access review, and periodic audit.
- Protect backup files because exported payloads can include sensitive institutional records.
- Enable HTTPS and secure cookies in deployment.

OPERATIONAL LOGIC

Module-by-Module Technical Behavior

Administrator Module

The administrator area is the governance center of the portal. It manages student records, faculty approval, support requests, attendance review, marks review, subject imports, audit logs, backup and restore, report cards, departmental analytics, and institution-level settings.

Administrative reports compute department-wise totals and performance indicators using SQL aggregates across attendance, marks, and assignment data. Settings include academic-year rollover, mark type management, mark locks, and full reset actions. Backup and restore operate across a fixed list of tables with foreign-key checks disabled during restore so entire datasets can be reloaded consistently.

Faculty Module

Faculty users work primarily inside attendance, marks, assignments, reports, students, and profile pages. Attendance is saved against class scope, date, and teacher. Marks can be entered manually or imported from CSV; invalid over-limit marks are normalized to zero in the CSV parsing path and surfaced back for review. Assignments are tracked per subject and assignment label, with student-level submission statuses.

The faculty reports and records pages allow broader review without changing the core academic data model. Profile pages support photo and contact updates, while recovery requests are routed to the support request queue for administrative review.

Student Module

Student accounts activate only from existing roster rows, which helps preserve institutional control over enrollment identity. Once activated, students can update contact details and profile image, monitor attendance percentage and daily history, review marks subject-wise, and track assignment submission status.

The student interface is intentionally narrower than the admin and faculty interfaces. It is designed for read-mostly self-service rather than data administration, which reduces risk while improving transparency for learners.

USER MANUAL

Role-Wise User Manual

Administrator Manual

1. Login through the admin login page using administrator credentials.
2. Review the dashboard for pending faculty approvals, departmental stats, and current operational summaries.
3. Use the Students page to add students manually, import students by CSV, filter rosters, or delete one or multiple students using available controls.
4. Open the Faculty page to approve, reject, edit, or remove faculty records.
5. Open the Requests page to review request, feedback, and issue records; approve or reject them as needed.
6. Use Subjects to import or manage semester-specific subject registers.
7. Use Attendance, Marks, Reports, and Report Cards to monitor academic output across departments.
8. Use Backup and Restore before major resets or academic-year transitions.
9. Use Settings to manage mark types, lock mark entry, create a new academic year, or perform reset operations very carefully.

Faculty Manual

1. Register with department, email, and password if an account does not yet exist.
2. Wait for administrative approval before attempting to use operational pages.
3. Use Mark Attendance to load class scope by department, year, semester, and date, then save the attendance sheet.
4. Use Attendance Records to review and revise existing class sessions and manage holiday entries where permitted.
5. Use Upload Marks to choose department, semester, subject, and mark type; then save marks manually or through the downloadable CSV template.
6. Use Assignment Tracking to mark submitted or pending status for each student per assignment label.
7. Use Reports and View Marks to review stored data across filters.
8. Use My Profile to update contact information, photo, and password where applicable.
9. If access is lost, use the faculty recovery flow so the request reaches the admin queue.

Student Manual

1. Activate the account only if the enrollment number already exists in the seeded institutional roster.
2. Login with enrollment number and password after activation.
3. Use the dashboard to view personal summary statistics and update contact information or profile image.
4. Open My Attendance to see percentage, recent status, and full attendance history.
5. Open My Marks to view subject-wise performance across all published mark components.
6. Open My Assignments to track submission progress for each assignment label.

Operational caution

Administrative reset, restore, bulk delete, and lock-management actions are intentionally powerful. These should be restricted to trained users, documented in SOPs, and preceded by a current backup.

MAINTENANCE

Backup, Restore, Academic-Year Rollover, and Maintenance Practice

Backup and Restore Lifecycle

Backup Export

- Collects rows from the configured backup table list.
- Packages the dataset into a JSON-compatible payload with generation metadata.
- Useful before destructive operations, deployment changes, or academic-year resets.

Restore Import

- Validates payload structure.
- Temporarily disables foreign-key checks.
- Truncates backup-managed tables in reverse order.
- Reinserts rows table by table and re-enables keys.

Recommended Administrative SOP

1. Export a backup and store it safely outside the live server.
2. Document the reason for any reset, restore, or rollover action.
3. Perform the change during a low-traffic window.
4. Verify student count, faculty count, current academic year, and sample reports after the operation.
5. Review the audit log and operational statistics after completion.

Academic-Year Transition Considerations

- Create the new academic year record and ensure only one year remains current.
- Decide whether historical records will be retained fully, partially cleared, or reset according to institutional policy.
- Revalidate subject lists, mark types, and department-wise locks after the rollover.

TESTING AND RISK

Testing Strategy, Residual Risks, and Troubleshooting

Recommended Test Matrix

AREA	MINIMUM VALIDATION	EXPECTED RESULT
Admin login	Correct and incorrect password attempts	Correct credentials start a session; incorrect ones do not
Faculty approval	Register, keep pending, approve, reject	Dashboard access changes according to status
Student activation	Use valid and invalid enrollment numbers	Only seeded roster records activate
Attendance	Save and re-open the same class/date	Session is updated, not duplicated
Marks CSV	Upload values over max, absent rows, missing entries	Violations are surfaced and stored according to rules
Assignments	Toggle submitted states	Status persists and shows in student view
Backup restore	Export, modify sandbox data, restore	Dataset returns to the exported state
Support requests	Create request, review from admin queue	Status changes and queue indicators update

Current Technical Risks

- No dedicated automated test suite is visible in the project snapshot, so quality depends heavily on manual verification.
- OTP delivery functions require real mail integration before the password recovery experience can be considered production-ready.
- Backup restore is intentionally destructive to current data; misuse can overwrite a live environment.
- As a server-rendered monolith, the application is maintainable for small teams but may need architectural evolution for large-scale concurrency or API expansion.

Troubleshooting Guide

Common Issues

Administrative Checks

- Review application logs in storage.

- Faculty cannot login after registration: check approval status.
- Student cannot activate: verify seeded roster row and matching department.
- Marks cannot save: check mark lock status and subject-selection scope.
- Upload rejected: verify CSV format, size, and allowed columns.

- Check audit log entries for action traces.
- Confirm current academic year is set correctly.
- Validate directory permissions for cache, logs, and uploads.

FUTURE SCOPE

Recommended Additional Features and Evolution Roadmap

To turn the portal from a strong internal academic management system into a more complete institutional platform, the following features are recommended as next-stage enhancements.

Notification Engine

Email and SMS alerts for approvals, attendance risk, marks publication, and assignment deadlines.

Soft Delete and Archive Mode

Safer record retirement with reversible archive states for students and faculty instead of only hard deletion.

Evidence Uploads

Assignment attachments, faculty document uploads, and downloadable student records.

API Layer

A REST or internal API abstraction for future mobile apps, analytics integrations, or ERP connections.

Advanced RBAC

Finer-grained permissions for sub-admins, department heads, and read-only reviewers.

Monitoring and Scheduler

Automated backups, alerting, retention rules, and background task execution for operational maturity.

Priority Recommendation

The highest-value next steps are production-grade reset-delivery integration, scheduled backup automation, soft-delete archival, and a stronger test strategy. These improvements would raise reliability without disrupting the current application design.

CONCLUSION

Final Assessment

BIPE Academic Portal is a well-scoped academic operations platform that combines administrative control with day-to-day usability. Its biggest strengths are role separation, workflow coverage, normalized academic data storage, and practical governance features such as audit logs, backup and restore, mark locks, and request review. The portal is especially suitable for small to medium academic environments where a single integrated web application can replace fragmented spreadsheets and manual follow-up.

From a technical perspective, the project is maintainable because shared logic is centralized, schema relationships are clear, and modules are grouped by role. From a non-technical perspective, it improves accountability, visibility, and turnaround time for routine academic processes. With stronger automated testing and production-grade mail integration, it can mature further into a robust institutional platform.

Prepared as a credential-free documentation artifact for the current BIPE Academic Portal codebase snapshot. All operational examples are descriptive and intentionally exclude confidential values.